czii-cryo-et-object-identification

1st place solution [segmentation with partly U-NET and ensembling part]

Rank	1
Team	Daddies
Author	Dieter
Topic ID	561510
Version	Original

Source: https://www.kaggle.com/competitions/czii-cryo-et-object-identification/discussion/561510
Retrieved with Kaggle CLI on 2026-10-21.

Thanks to kaggle and everyone involved for hosting this exciting competition. It was a great learning experience and it was very interesting to see how much of our computer vision experience could also be applied to 3D imaging. Thanks to @bloodaxe for this great team experience.

TLDR

The solution in an ensemble of segmentation (3D Unets with ResNet & B3 encoders) and object detection models (SegResNet and DynUnet backbones) from MONAI. We also used MONAI for augmentations, and exported models via jit or TensorRT, which gave 200% speedup increase and enabled us to have a slightly larger ensemble. We did not use any external or simulated data!

This post covers the segmentation based approach and ensembling. For object detection part see @bloodaxe writeup: 1st place solution [Object Detection Part]

Cross validation

For segmentation approach 7 folds were used, simply by splitting by experiment. Using mean of f4-score of all 7 folds had some good correlation with LB. During model training I optimized individual class thresholds at the end of each epoch by simple grid-search on the validation experiment. After all 7 folds were trained we could re-calibrate thresholds, by taking OOF predictions. And fitting the threshold for one fold on the predictions of the other 6. Then we average the resulting f4-curves and take the best threshold.

Data preprocessing/ augmentations

3D images were normalized by standard normalization, i.e. for each 630x630x184 image, we substract mean and devide by standard deviation before splitting the images into patches.
Since models are trained from scratch, augmentations were essential to prevent overfitting.
We used RandomCrop, Flip on each axis, and rotation, which all are available with MONAI. Additionally I used my own implementation of MixUp which was highly effective to train longer and prevent overfitting.

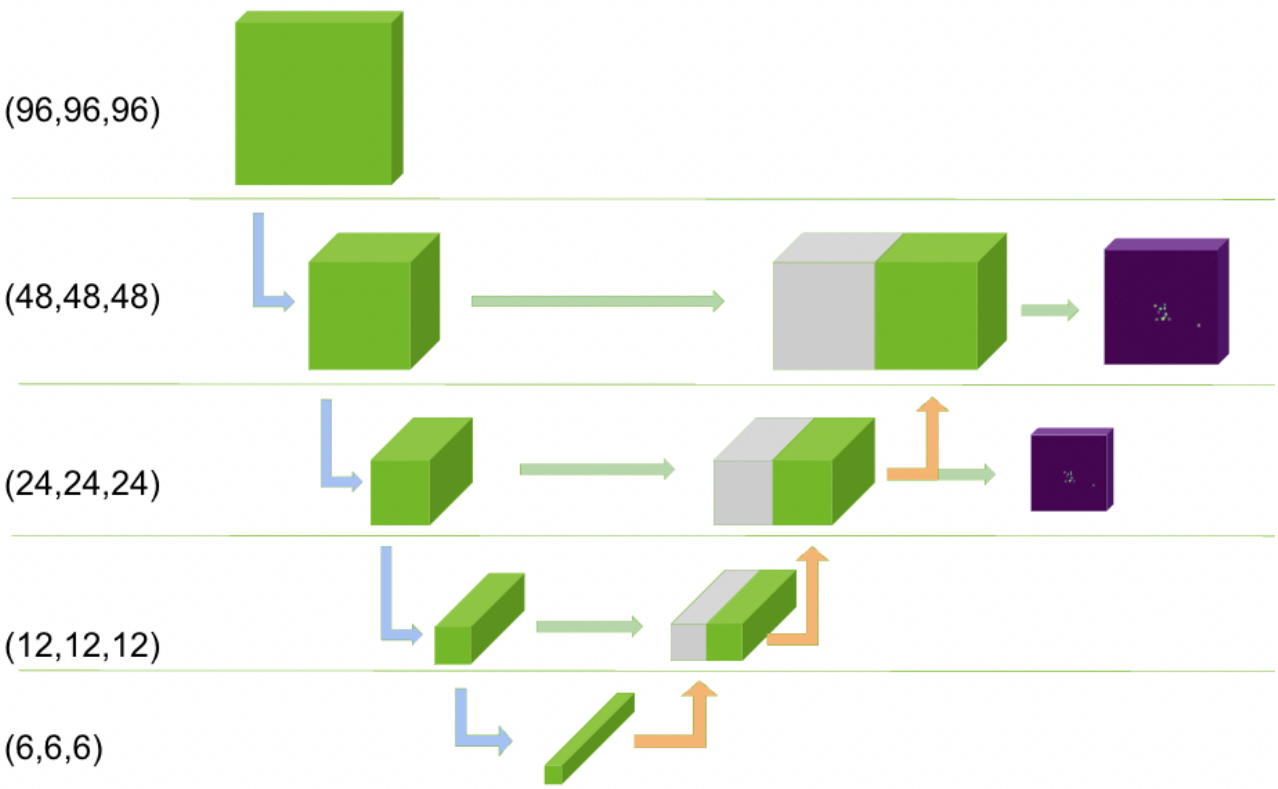
Model

Modelling was quite a ride in this competition. I started with simple UNET with having 3D gaussian balls as segmentation target. For diversity I also tried object detection example from MONAI and realized its working very well out of the box. But when analyzing the different output feature maps and trying to isolate the perfomance gain over gaussian heatmap based segmentation I realized where the advantage was and adjusted my segmentation model accordingly. I learned that

- the penultimate feature map has a higher accuracy than the last one, which is surprising at first
- gains from box regression are negligable, as particles from the same type have mostly same size anyways.

Hence its sufficient to have a pixel-wise loss on the penultimate feature map output. I.e. use a **partly** UNET. The gaussian heatmap is not needed when suppressing background with a low class weight, and just use single pixels as targets. Using the same approach on lower level outputs (= deep supervision) is possible, but

does not provide much gain. Input for the segmentation models are 96x96x96 image patches and the loss is calculated on the 48x48x48 output.



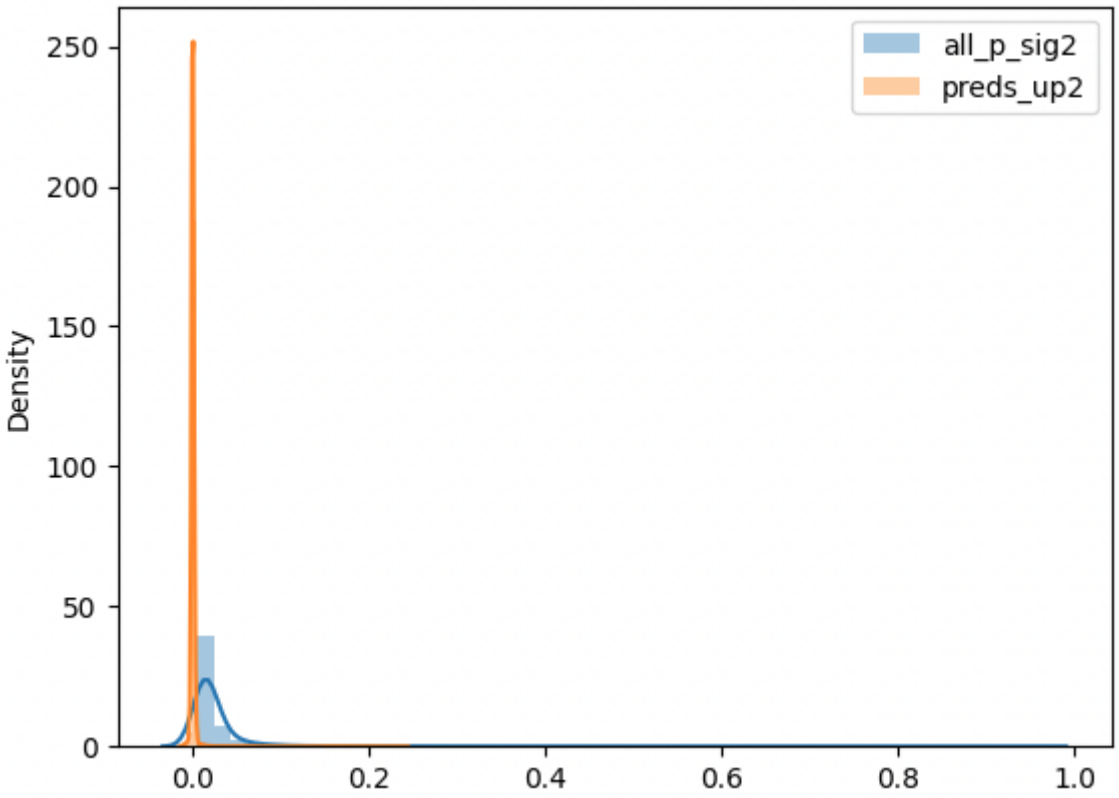
In general, we observed that relatively small models work really and the design of loss is the most important aspect. We used MONAIs FlexibleUnet with backbones resnet34 and efficientnet-b3. Six checkpoints of this architecture would finish under 2h and score 7th place on LB.

Training procedure

We used 7 classes (incl background) and weighted CrossEntropy as loss. Notably keeping beta-amylase as a class although it is not scored is quite helpful as model learns to differentiate beta-galactosidase from it. To account for low number of positive pixels, positive pixels are weighted by 256 and background has weight 1. Models were trained with a cosine learning rate schedule with peak LR of 0.001, mixed precision and an effective batch size of 32 samples. Training is based on Random crops and for validation the single experiment image is divided into patches and stored in RAM.

Ensembling

Ensembling was very challenging, as our two approaches are quite diverse. While in theory predictions from the segmentation models can be ensembled with feature map outputs of the object detection model before runnning the object detection postprocessing, in practice those feature maps have a very different distribution due to difference architectures and loss functions.

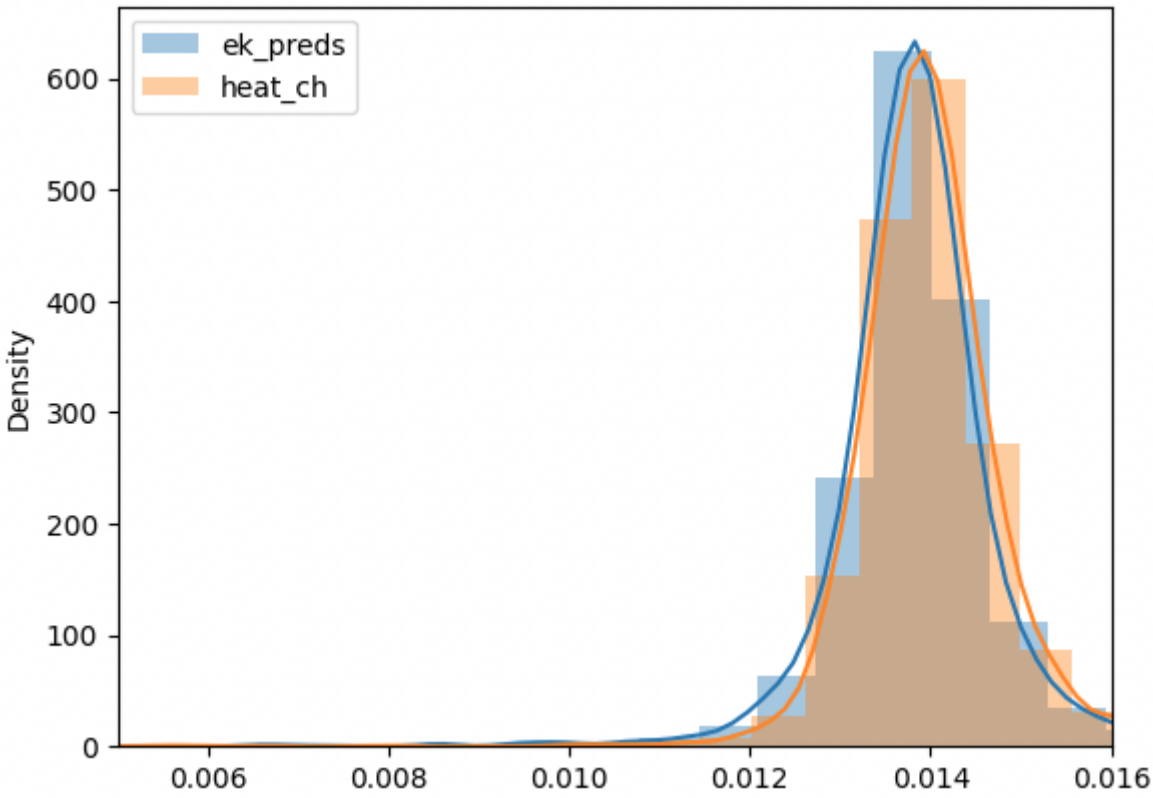


We were eager to find an elegant way to fix this scaling issue as we saw the potential of a possible ensemble. The scaling of our best submission works as following. To combine predictions A with predictions B, for each class sort all pixel values for A and B and replace values of B with the corresponding values of A of same rank. In code:

```
# rank scaling per class

adj_preds_B = torch.zeros_like(preds_B)
for c in range(n_classes):
    q = preds_B[c].flatten().argsort().argsort()
    r = preds_A[c].flatten()
    r2 = r.argsort()
    adj_preds_B[c] += r[r2[q]].reshape(preds_B[c].shape)
```

This results in both predictions having the same distribution, and hence we could simple blend the feature maps before performing object de
tection task.



What did not work

- Using supplemental data (external or simulated)
- Other augs
- Other losses (Tversky, Dice)

Thanks for reading.

Edits:
training code: https://github.com/ChristofHenkel/kaggle-cryoet-1st-place-segmentation
inference kernel: https://www.kaggle.com/code/christofhenkel/cryo-et-1st-place-solution?scriptVersionId=223259615

Supplemental Q&A

> **Ma Edward · 2025-02-06T14:17:42.937000**
>
> Congratulations!!!
> Would you mind sharing the single model performance in the final ensemble models?

> **Dieter · 2025-02-06T14:25:51.720000**
>
> for segmentation:
>
> resnet34 backbone: 0.766
> resnet34 backbone + deep supervision: 0.767

effnet-b3 backbone: 0.765
combined: 0.778

Bartley · 2025-02-06T17:58:23.737000

Thanks for the detailed write-up and nice ensemble strategy.

How was your version of MixUp different from the usual implementation?

Đăng Nguyễn Hồng · 2025-02-07T07:19:16.023000

Congratulations and thank you so much for your write-up, as always!
The CV scheme truly impressive. The ensemble approach seems like a form of histogram matching, isn't it?
For single model, it's remarkable how your team and the top solutions have made everything work so seamlessly, especially when I struggled to achieve just a poorer result with a "sound-like similar" setup. There are likely many implementation details that come naturally to you but still pose a bit of a challenge for me.
Really looking forward to seeing your train/infer code so I can better understanding of our problems and failures!

Yassine Alouini · 2025-02-07T19:45:43.580000

Well done and congrats. Can you provide some details on the training infrastructure please (how many GPUs, how long it trained, any issues on the losses…)?

Dieter · 2025-02-07T20:22:59.030000

Its not much different to other implementations. Some mix rather the losses, I like mixing the targets better. I like a simple pure torch implementation so you can have it as part of the model on GPU. And its flexible for 1D, 2D, 3D.

```
torch
torch  nn
torch.nn.functional  F
torch.nn.parameter  Parameter
torch.distributions  Beta

(nn.Module):
    ():

       (Mixup, ).__init__()
       .beta_distribution = Beta(mix_beta, mix_beta)
       .mixadd = mixadd

    ():

       bs = X.shape[]
       n_dims = (X.shape)
       perm = torch.randperm(bs)
       coeffs = .beta_distribution.rsample(torch.Size((bs,))).to(X.device)
       X_coeffs = coeffs.view((-,) + (,)*(X.ndim-))
       Y_coeffs = coeffs.view((-,) + (,)*(Y.ndim-))

       X = X_coeffs * X + (-X_coeffs) * X[perm]

        .mixadd:
           Y = (Y + Y[perm]).clip(, )
       :
```

```
            Y = Y_coeffs * Y + ( - Y_coeffs) * Y[perm]

         Z:
             X, Y, Z

         X, Y


(nn.Module):

    ():
       (Net, ).__init__()

       ...
       .mixup = Mixup(cfg.mixup_beta)
       ...

    ():

       x = batch[]
       y = batch[]
        .training:
            torch.rand()[] < .cfg.mixup_p:
               x, y = .mixup(x,y)
       ....
```

Yassine Alouini · 2025-02-08T17:22:50.443000

I guess he adapted an existing implementation to work with 3D patches? As far as my knowledge goes, MixUp works with two images with weights alpha and (1-alpha), maybe he did something clever to have images over the temporal dimension?

Dieter · 2025-02-09T11:22:02.583000

I trained 6 checkpoints, each takes 2h on a A100 with bf16 mixed-precision. For my models mixup was crucial which more than doubles the number of epochs for training.

Dieter · 2025-02-09T11:30:19.753000

I was already 1st place on public LB two months ago, with a simple 3D-Unet with gaussian balls as target. Score was around 0.74 back then. It took us 2 month of hard work to improve and refine so the final solution can look "seamlessly" as you say.

Dieter · 2025-02-09T11:45:57.913000

Mixup really just is averaging inputs and targets of a batch with a weight drawn from a Beta-distribution. Actually we (@philippsinger and @ilu000) coded this four years ago as part of the rainforest competition if I recall correctly. Mixup is really strong for audio. Our implementation was already flexible for arbitrary dimension input and was running on GPU, and I re-used in several competitions since then.